



# Errors2Experts IT Training & Services

Thamaraipadi, Dindigul, Tamil Nadu

## Python Programming

### Intermediate Topics Guide

User Input • Strings • String Methods • Loops • OOP • Decorators

|    |                |                                                        |
|----|----------------|--------------------------------------------------------|
| 01 | User Input     | input(), type casting, validation                      |
| 02 | Strings        | Creation, slicing, immutability                        |
| 03 | String Methods | All built-in methods with examples                     |
| 04 | Loops          | for, while, nested, break, continue                    |
| 05 | OOP            | Classes, objects, methods, encapsulation, polymorphism |
| 06 | Decorators     | @decorator, functools, practical use cases             |

5 Programs + 5 Tasks per Topic | Easy to Understand | Errors2Experts

# TOPIC 1: USER INPUT

## What is User Input?

The **input()** function allows your program to receive data from the user while it is running. Whatever the user types is always returned as a **string**. You must convert it to **int** or **float** when you need numbers.

```
name = input(Enter your name: )
age = int(input(Enter your age: ))
print(Hello, name, Age: , age)
```

■ *input() always returns a string. Use int() or float() to convert to numbers.*

## Type Conversion with input

- `int(input())` — convert to whole number
- `float(input())` — convert to decimal
- `str(input())` — already string, but explicit
- `list(input())` — converts each character to list item

## 5 Programs — User Input

### Program 1: Basic Name and Age Input

```
name = input(Enter your name: )
age = int(input(Enter your age: ))
print(Hello {}{} .format(name, age))
```

#### Output:

```
Enter your name: Ravi
Enter your age: 21
Hello Ravi 21
```

### Program 2: Simple Calculator with Input

```
a = float(input(Enter first number: ))
b = float(input(Enter second number: ))
print(Sum: , a+b)
print(Diff: , a-b)
print(Product:, a*b)
print(Divide: , a/b)
```

#### Output:

```
Enter first number: 10
Enter second number: 3
Sum: 13.0
Diff: 7.0
Product: 30.0
```

Divide: 3.333...

### Program 3: Check Even or Odd with Input

```
n = int(input(Enter a number: ))
if n % 2 == 0:
    print(n, is Even)
else:
    print(n, is Odd)
```

#### Output:

```
Enter a number: 7
7 is Odd
```

### Program 4: Multi-line Input using split()

```
# Take 3 numbers in one line: 10 20 30
a, b, c = map(int, input(Enter 3 numbers: ).split())
print(Numbers: , a, b, c)
print(Sum: , a+b+c)
```

#### Output:

```
Enter 3 numbers: 10 20 30
Numbers: 10 20 30
Sum: 60
```

### Program 5: Input Validation with Loop

```
while True:
    age = int(input(Enter age (1-120): ))
    if 1 <= age <= 120:
        print(Valid age: , age)
        break
    else:
        print(Invalid! Try again.)
```

#### Output:

```
Enter age (1-120): -5
Invalid! Try again.
Enter age (1-120): 21
Valid age: 21
```

---

## 5 Tasks — User Input

| Task | Description                                                                                                      |
|------|------------------------------------------------------------------------------------------------------------------|
| 1    | Take name, city, and course from user. Print a formatted message: 'Welcome Ravi from Dindigul to Python course!' |

Task  
2

Take two numbers from user. Print their sum, difference, product, and division.

Task  
3

Take a student's 5 subject marks using input() in a loop. Calculate and print total and average.

Task  
4

Take a number from user and print its multiplication table from 1 to 10.

Task  
5

Build a simple login: ask username and password. If username='admin' and password='1234', print 'Welcome Admin'. Otherwise 'Wrong credentials'.

## ■ TOPIC 2: STRINGS

### ■ What is a String?

A **string** is a sequence of characters. Strings are **immutable** — once created, you cannot change them. Python supports single quotes, double quotes, and triple quotes. Strings are indexed starting from 0.

```
s1 = 'Hello' # single quotes
s2 = "World" # double quotes
s3 = '''Multi line''' # triple quotes
s4 = "I'm learning" # mix quotes
```

### ■ String Indexing and Slicing

Each character has an index. Positive index starts from 0 (left). Negative index starts from -1 (right). Slicing syntax: **string[start:stop:step]**

```
s = "Python"
# P y t h o n
# index 0 1 2 3 4 5
# neg -6 -5 -4 -3 -2 -1

print(s[0]) # P - first character
print(s[-1]) # n - last character
print(s[0:3]) # Pyt - first 3
print(s[::-1]) # nohtyP - reverse
print(s[2::2]) # ton - every 2nd from index 2
```

### ■ String is Immutable

■ You CANNOT change a character of a string directly. `s[0] = 'J'` gives `TypeError`. Instead, create a new string.

## 5 Programs — Strings

### Program 1: String Creation and Access

```
s = "Errors2Experts"
print(s[0]) # E
print(s[-1]) # s
print(s[0:6]) # Errors
print(s[7:]) # Experts
print(len(s)) # 14
```

#### Output:

```
E
s
Errors
```

Experts

14

## Program 2: String Repetition and Concatenation

```
a = "Hello"
b = "Python"
print(a + " " + b) # concatenation
print(a * 3) # repetition
print(a + "!" * 5)
```

### Output:

```
Hello Python
HelloHelloHello
Hello!!!!!
```

## Program 3: String Slicing Tricks

```
s = "ABCDEFGHJIJ"
print(s[:5]) # first 5: ABCDE
print(s[5:]) # from 5: FGHIJ
print(s[::-1]) # reverse: JIHGFEDCBA
print(s[::2]) # every 2nd: ACEGI
print(s[2:8:2]) # CEGI
```

### Output:

```
ABCDE
FGHIJ
JIHGFEDCBA
ACEGI
CEGI
```

## Program 4: String Formatting — f-strings

```
name = "Ravi"
age = 21
marks= 92.5
print(f"Name: {name}, Age: {age}")
print(f"Marks: {marks:.1f}%")
print(f"{name} scored {marks}")
```

### Output:

```
Name: Ravi, Age: 21
Marks: 92.5%
RAVI scored 92.5
```

### Program 5: Check Characters in String

```
s = "Python Programming"
print("Python" in s) # True
print("Java" in s) # False
print(s.startswith("Py"))
print(s.endswith("ing"))
for ch in s[:6]: print(ch, end="-")
```

#### Output:

```
True
False
True
True
P-y-t-h-o-n-
```

---

### 5 Tasks — Strings

- |        |                                                                                         |
|--------|-----------------------------------------------------------------------------------------|
| Task 1 | Take your full name as a string. Print first name, last name, and full name in reverse. |
| Task 2 | Take any string. Print: length, first character, last character, middle character.      |
| Task 3 | Check if the string 'Errors2Experts' contains '2'. Print True or False.                 |
| Task 4 | Take a sentence from user. Count total characters (with spaces) and without spaces.     |
| Task 5 | Print every alternate character of the string 'PythonProgramming' using slicing.        |

## TOPIC 3: STRING METHODS

### What are String Methods?

Python strings come with many built-in methods that make it easy to manipulate text. Since strings are immutable, all methods **return a new string** — they don't modify the original.

| Method                     | What it does                   | Example                                      | Output          |
|----------------------------|--------------------------------|----------------------------------------------|-----------------|
| <code>upper()</code>       | All uppercase                  | <code>"hello".upper()</code>                 | HELLO           |
| <code>lower()</code>       | All lowercase                  | <code>"HELLO".lower()</code>                 | hello           |
| <code>title()</code>       | Title case                     | <code>"hello world".title()</code>           | Hello World     |
| <code>capitalize()</code>  | First letter upper             | <code>"python".capitalize()</code>           | Python          |
| <code>strip()</code>       | Remove leading/trailing spaces | <code>" hi ".strip()</code>                  | hi              |
| <code>lstrip()</code>      | Remove left spaces             | <code>" hi ".lstrip()</code>                 | hi              |
| <code>rstrip()</code>      | Remove right spaces            | <code>" hi ".rstrip()</code>                 | hi              |
| <code>replace(a,b)</code>  | Replace text                   | <code>"hi bye".replace("hi", "hello")</code> | hello bye       |
| <code>split(sep)</code>    | Split to list                  | <code>"a,b,c".split(",")</code>              | ['a', 'b', 'c'] |
| <code>join(list)</code>    | Join list to string            | <code>"-".join(["a", "b"])</code>            | a-b             |
| <code>find(s)</code>       | Find index (–1 if missing)     | <code>"hello".find("l")</code>               | 2               |
| <code>index(s)</code>      | Find index (error if missing)  | <code>"hello".index("l")</code>              | 2               |
| <code>count(s)</code>      | Count occurrences              | <code>"hello".count("l")</code>              | 2               |
| <code>startswith(s)</code> | Check start                    | <code>"hello".startswith("he")</code>        | True            |
| <code>endswith(s)</code>   | Check end                      | <code>"hello".endswith("lo")</code>          | True            |
| <code>isdigit()</code>     | All digits?                    | <code>"123".isdigit()</code>                 | True            |
| <code>isalpha()</code>     | All letters?                   | <code>"abc".isalpha()</code>                 | True            |
| <code>isalnum()</code>     | Letters or digits?             | <code>"abc123".isalnum()</code>              | True            |
| <code>isspace()</code>     | All spaces?                    | <code>" ".isspace()</code>                   | True            |
| <code>isupper()</code>     | All uppercase?                 | <code>"HELLO".isupper()</code>               | True            |
| <code>islower()</code>     | All lowercase?                 | <code>"hello".islower()</code>               | True            |
| <code>zfill(n)</code>      | Pad with zeros                 | <code>"42".zfill(5)</code>                   | 00042           |
| <code>center(n,c)</code>   | Center with fill               | <code>"hi".center(10, "-")</code>            | ----hi----      |
| <code>ljust(n)</code>      | Left justify                   | <code>"hi".ljust(5, ".")</code>              | hi...           |
| <code>rjust(n)</code>      | Right justify                  | <code>"hi".rjust(5, ".")</code>              | ...hi           |
| <code>encode()</code>      | Encode string                  | <code>"hi".encode()</code>                   | b'hi'           |
| <code>format()</code>      | Format string                  | <code>"Hi {}".format("Ravi")</code>          | Hi Ravi         |
| <code>swapcase()</code>    | Swap upper/lower               | <code>"Hello".swapcase()</code>              | hELLO           |



## 5 Programs — String Methods

### Program 1: Case Methods

```
s = "hello world python"
print(s.upper())
print(s.title())
print(s.capitalize())
print(s.swapcase())
```

#### Output:

```
HELLO WORLD PYTHON
Hello World Python
Hello world python
HELLO WORLD PYTHON
```

### Program 2: Search and Replace Methods

```
s = " Python is easy and Python is fun "
print(s.strip())
print(s.count("Python"))
print(s.find("easy"))
print(s.replace("Python","Java"))
```

#### Output:

```
Python is easy and Python is fun
2
10
Java is easy and Java is fun
```

### Program 3: Split, Join and Strip

```
csv = "Ravi,Kumar,Priya,Siva,Mala"
names = csv.split(",")
print(names)
print(len(names))
print(" | ".join(names))
```

#### Output:

```
['Ravi', 'Kumar', 'Priya', 'Siva', 'Mala']
5
Ravi | Kumar | Priya | Siva | Mala
```

### Program 4: Validation Methods

```
print("12345".isdigit())
```

```
print("hello".isalpha())  
print("hello123".isalnum())  
print("PYTHON".isupper())  
print("python".islower())
```

**Output:**

```
True  
True  
True  
True  
True
```

**Program 5: Padding and Alignment**

```
s = "Errors2Experts"  
print(s.center(30, "*"))  
print(s.ljust(25, "-"))  
print(s.rjust(25, "-"))  
print("42".zfill(8))
```

**Output:**

```
*****Errors2Experts*****  
Errors2Experts-----  
Errors2Experts  
00000042
```

**5 Tasks — String Methods**

- |        |                                                                                                                  |
|--------|------------------------------------------------------------------------------------------------------------------|
| Task 1 | Take a sentence from user. Print it in uppercase, title case, and swapped case.                                  |
| Task 2 | Take any string. Count how many vowels (a,e,i,o,u) are in it using count().                                      |
| Task 3 | Take a comma-separated list of names: 'Ravi,Kumar,Priya'. Split it and print each name on a new line.            |
| Task 4 | Take a user's email address. Check if it ends with '@gmail.com'. Print 'Valid' or 'Invalid'.                     |
| Task 5 | Take a number as string (e.g. '42'). Pad it with zeros to make it 8 digits using zfill(). Check if it isdigit(). |

## ■ TOPIC 4: LOOPS — DEEP DIVE

### ■ Types of Loops in Python

Python has two loop types: **for loop** (iterates over a sequence) and **while loop** (runs based on a condition). Loops can be controlled using **break**, **continue**, and **pass**.

### ■ for loop with range()

```
for i in range(5): # 0,1,2,3,4
for i in range(1,6): # 1,2,3,4,5
for i in range(0,10,2): # 0,2,4,6,8
for i in range(10,0,-1): # 10,9,8...1
```

### ■ while loop

```
i = 0
while i < 5:
    print(i)
    i += 1
```

### ■ Loop Control — break, continue, pass

- **break** — exits the loop completely immediately
- **continue** — skips current iteration, jumps to next
- **pass** — placeholder, does nothing, loop continues
- **else** — runs after loop finishes normally (not after break)

### ■ enumerate() and zip()

```
# enumerate gives index + value
for idx, val in enumerate(["A","B","C"]):
    print(idx, val) # 0 A, 1 B, 2 C

# zip combines two lists
for n,m in zip(["Ravi","Kumar"],[90,85]):
    print(n,m) # Ravi 90, Kumar 85
```

---

## 5 Programs — Loops

### Program 1: for Loop Patterns

```
# Sum of 1 to 100
total = 0
for i in range(1,101):
    total += i
print(Sum 1 to 100: , total)
```

```
# Factorial of 5

fact = 1

for i in range(1,6): fact *= i

print(5! = , fact)
```

**Output:**

```
Sum 1 to 100: 5050

5! = 120
```

**Program 2: while Loop — Guess the Number**

```
import random

secret = random.randint(1,10)

while True:

    guess = int(input(Guess 1-10: ))

    if guess == secret:

        print(Correct!); break

    elif guess < secret:

        print(Too low!)

    else:

        print(Too high!)
```

**Output:**

```
Guess 1-10: 5

Too low!

Guess 1-10: 8

Correct!
```

**Program 3: Nested Loop — Pattern Printing**

```
# Print right triangle

for i in range(1,6):

    for j in range(i):

        print(*, end=" ")

    print()
```

**Output:**

```
*

* *

* * *

* * * *

* * * * *
```

**Program 4: break, continue, else**

```
# continue – skip even numbers
```

```
for i in range(1,11):
    if i%2 == 0: continue
    print(i, end=" ")
    print()

# break – stop at first multiple of 7
for i in range(1,50):
    if i%7 == 0: print(First mult of 7: , i); break
```

**Output:**

```
1 3 5 7 9
First mult of 7: 7
```

**Program 5: enumerate() and zip() in Action**

```
subjects = ["Math","Science","English"]
marks = [90,85,78]

print(Report Card:)
for i,(sub,mark) in enumerate(zip(subjects,marks),1):
    print(f"{i}. {sub}: {mark}")
```

**Output:**

```
Report Card:
1. Math: 90
2. Science: 85
3. English: 78
```

---

**5 Tasks — Loops**

- |        |                                                                                                                   |
|--------|-------------------------------------------------------------------------------------------------------------------|
| Task 1 | Print multiplication table of any number the user enters (1 to 12).                                               |
| Task 2 | Using a while loop, keep asking the user for a password until they type 'python123'. Then print 'Access Granted'. |
| Task 3 | Print all prime numbers between 1 and 50 using nested loops.                                                      |
| Task 4 | Using enumerate(), print all items in a list with their index number starting from 1.                             |
| Task 5 | Take a list of 5 student names and 5 marks. Use zip() to print 'Ravi: 90, Kumar: 85...' for each.                 |

# TOPIC 5: OBJECT ORIENTED PROGRAMMING

## 4 Pillars of OOP

OOP organizes code around real-world objects. Python is fully object-oriented. The 4 pillars are: **Encapsulation**, **Abstraction**, **Inheritance**, and **Polymorphism**.

| Pillar        | Meaning                                                                              | Real Life Example                               |
|---------------|--------------------------------------------------------------------------------------|-------------------------------------------------|
| Encapsulation | Bundling data + methods together, <del>Admin</del> <del>Admin</del> <del>Admin</del> | Admin: you use it without knowing how it works  |
| Abstraction   | Showing only necessary details                                                       | Car: you drive without knowing engine internals |
| Inheritance   | Child class gets properties of parent class                                          | Class: inherits father's traits                 |
| Polymorphism  | Same method works differently in different classes                                   | Dog barks, Cat meows                            |

## Class Anatomy

```
class Student:
    # Class variable - shared by all objects
    school = "Errors2Experts"

    def __init__(self, name, marks): # constructor
        self.name = name # instance variable
        self.marks = marks

    def info(self): # instance method
        return f"{self.name}: {self.marks}"

    def __str__(self): # string representation
        return self.info()

s1 = Student("Ravi", 90) # create object
print(s1) # Ravi: 90
```

## Access Modifiers

- Public: self.name — accessible everywhere
- Protected: self.\_name — convention, accessible but should not be from outside
- Private: self.\_\_name — name mangled, truly restricted

## 5 Programs — OOP

### Program 1: Encapsulation — Bank Account

```
class BankAccount:
    def __init__(self, owner):
        self.owner = owner
        self.__balance = 0 # private
```

```
def deposit(self, amt):
    if amt > 0: self.__balance += amt

def withdraw(self, amt):
    if amt <= self.__balance: self.__balance -= amt
    else: print(Insufficient balance!)

def show(self):
    print(f"Balance: {self.__balance}")

acc = BankAccount("Kumar")

acc.deposit(5000); acc.withdraw(2000); acc.show()
```

**Output:**

```
Balance: 3000
```

**Program 2: Inheritance — Animal System**

```
class Animal:
    def __init__(self, name):
        self.name = name
    def eat(self): print(self.name, is eating)

class Dog(Animal):
    def bark(self): print(self.name, says: Woof!)

class Cat(Animal):
    def meow(self): print(self.name, says: Meow!)

d = Dog("Tommy"); d.eat(); d.bark()
c = Cat("Kitty"); c.eat(); c.meow()
```

**Output:**

```
Tommy is eating
Tommy says: Woof!
Kitty is eating
Kitty says: Meow!
```

**Program 3: Polymorphism — Same Method Different Behavior**

```
class Shape:
    def area(self): return 0

class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return 3.14 * self.r ** 2

class Rectangle(Shape):
    def __init__(self, l, b): self.l=l; self.b=b
    def area(self): return self.l * self.b
```

```
for shape in [Circle(5), Rectangle(4,6), Shape()]:  
    print(Area: , shape.area())
```

**Output:**

```
Area: 78.5  
Area: 24  
Area: 0
```

**Program 4: super() and Method Overriding**

```
class Person:  
    def __init__(self, name, age):  
        self.name = name; self.age = age  
    def info(self):  
        return f"{self.name} ({self.age})"  
  
class Employee(Person):  
    def __init__(self, name, age, dept, salary):  
        super().__init__(name, age)  
        self.dept = dept; self.salary = salary  
    def info(self): # override  
        return super().info() + f" | {self.dept} | Rs.{self.salary}"  
  
e = Employee("Senthil", 28, "IT", 50000)  
print(e.info())
```

**Output:**

```
Senthil (28) | IT | Rs.50000
```

**Program 5: Class and Static Methods**

```
class MathUtils:  
    pi = 3.14159  
  
    @classmethod  
    def circle_area(cls, r):  
        return cls.pi * r ** 2  
  
    @staticmethod  
    def add(a, b): return a + b  
  
print(MathUtils.circle_area(5))  
print(MathUtils.add(10, 20))
```

**Output:**

```
78.53975  
30
```



## 5 Tasks — OOP

|        |                                                                                                                               |
|--------|-------------------------------------------------------------------------------------------------------------------------------|
| Task 1 | Create a 'Student' class with name, roll, marks. Add method grade() that returns A/B/C/F based on marks.                      |
| Task 2 | Create 'Vehicle' parent class with brand, speed. Create 'Car' child with extra attribute fuel_type. Print all details.        |
| Task 3 | Create 'Library' class with private __books list. Add methods add_book(), remove_book(), show_books().                        |
| Task 4 | Create 3 classes: Shape, Triangle (area = $0.5*b*h$ ), Square (area = $s*s$ ). Demonstrate polymorphism.                      |
| Task 5 | Create an 'Employee' class with __salary (private). Add get_salary() and increment(percent) methods. Test with 20% increment. |

## ■ TOPIC 6: DECORATORS

### ■ What is a Decorator?

A **decorator** is a function that takes another function as input, adds extra functionality to it, and returns a new function. Decorators use the @ symbol. They are used for logging, timing, authentication, caching, and more.

### ■ How Decorators Work — Step by Step

```
# Step 1: Without decorator

def my_decorator(func):

    def wrapper():
        print(Before function runs)
        func() # call original function
        print(After function runs)
    return wrapper

# Step 2: Applying decorator

@my_decorator # same as: greet = my_decorator(greet)

def greet():
    print(Hello from greet!)

greet() # output: Before → Hello → After
```

■ *@my\_decorator before a function is shorthand for: greet = my\_decorator(greet)*

### ■ Decorator with Arguments using functools.wraps

```
from functools import wraps

def my_decorator(func):
    @wraps(func) # preserves original function metadata
    def wrapper(*args, **kwargs):
        print(f"Calling: {func.__name__}")
        result = func(*args, **kwargs)
        print(Done!)
        return result
    return wrapper
```

---

## 5 Programs — Decorators

### Program 1: Basic Decorator

```
def star_border(func):

    def wrapper():
        print(* * 30)

    func()
```

```
print(* * 30)

return wrapper

@star_border
def message():
    print( Welcome to Errors2Experts! )

message()
```

**Output:**

```
*****
Welcome to Errors2Experts!
*****
```

**Program 2: Timer Decorator — Measure Execution Time**

```
import time

from functools import wraps

def timer(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} took {end-start:.4f}s")
        return result
    return wrapper

@timer
def calculate():
    return sum(range(1000000))

print(calculate())
```

**Output:**

```
calculate took 0.0423s
499999500000
```

**Program 3: Logger Decorator**

```
from functools import wraps

def logger(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print(f"CALL: {func.__name__}{args}")
        result = func(*args, **kwargs)
```

```
print(f"RETURN: {result}")
return result
return wrapper

@logger
def add(a,b): return a+b

add(5,3)
```

**Output:**

```
CALL: add(5, 3)
RETURN: 8
```

**Program 4: Decorator with Parameters**

```
def repeat(n): # decorator factory
def decorator(func):
def wrapper(*args,**kwargs):
for _ in range(n):
func(*args,**kwargs)
return wrapper
return decorator

@repeat(3)
def say_hello():
print>Hello from E2E!

say_hello()
```

**Output:**

```
Hello from E2E!
Hello from E2E!
Hello from E2E!
```

**Program 5: Stacking Multiple Decorators**

```
def bold(func):
def wrapper(): return f"{func()}"
return wrapper

def italic(func):
def wrapper(): return f"{func()}"
return wrapper

@bold
@italic # applied bottom-up: italic first, then bold
def greet():
return Hello Errors2Experts
```

```
print(greet())
```

Output:

```
Hello Errors2Experts
```

---

## 5 Tasks — Decorators

- |        |                                                                                                                                                |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Task 1 | Write a decorator 'uppercase_result' that converts the return value of a function to uppercase. Apply it to a function that returns your name. |
| Task 2 | Write a 'login_required' decorator. If a variable is_logged_in=True, run the function. Otherwise print 'Please login first'.                   |
| Task 3 | Write a decorator that counts how many times a function has been called. Print 'Function called X times' each time.                            |
| Task 4 | Write a 'validate_age' decorator that checks if age argument is between 1-120. If not, print error. Apply to a register() function.            |
| Task 5 | Write a decorator factory '@retry(n)' that runs a function up to n times if it raises an exception. Test with a function that randomly fails.  |

## Quick Reference Summary

| Topic          | Key Points                                                                                            | Remember                                                                                              |
|----------------|-------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| User Input     | input() always returns string<br>Use int(), float() to convert<br>Validate with try-except or if      | Always validate user input<br>Handle wrong inputs gracefully                                          |
| Strings        | Immutable sequence of characters<br>Indexing: s[0], s[-1]<br>Slicing: s[start:stop:step]              | s[::-1] reverses string<br>f-strings are fastest formatting<br>Strings cannot be changed in place     |
| String Methods | upper/lower/title/strip/replace<br>split/join/find/count/startswith<br>isdigit/isalpha/isalnum        | Methods return new string<br>Chain methods: s.strip().lower()<br>strip() removes whitespace           |
| Loops          | for: iterate over sequence<br>while: run based on condition<br>break/continue/pass/else               | range(start,stop,step)<br>enumerate() gives index+value<br>zip() combines two lists                   |
| OOP            | 4 pillars: Encap, Abstrac, Inherit, Poly<br>class → __init__ → self<br>Inheritance with super()       | __init__ is constructor<br>self refers to current object<br>__ prefix makes attribute private         |
| Decorators     | Function that wraps another function<br>@decorator syntax<br>Use functools.wraps to preserve metadata | Can be stacked with multiple @<br>Decorator factory: def dec(arg)<br>Useful for logging, timing, auth |

Keep Learning. Keep Building. Keep Growing!

Errors2Experts IT Training & Services

Python Full Stack — ■5,000 | Frontend — ■4,000 | Python Basics — ■2,500

■ Thamarapadi, Dindigul, Tamil Nadu

■ 9363342646 | ✉ errors2experts.official@gmail.com

■ errors2experts.pythonanywhere.com